

CS620T: Compilers - Principles and Implementation

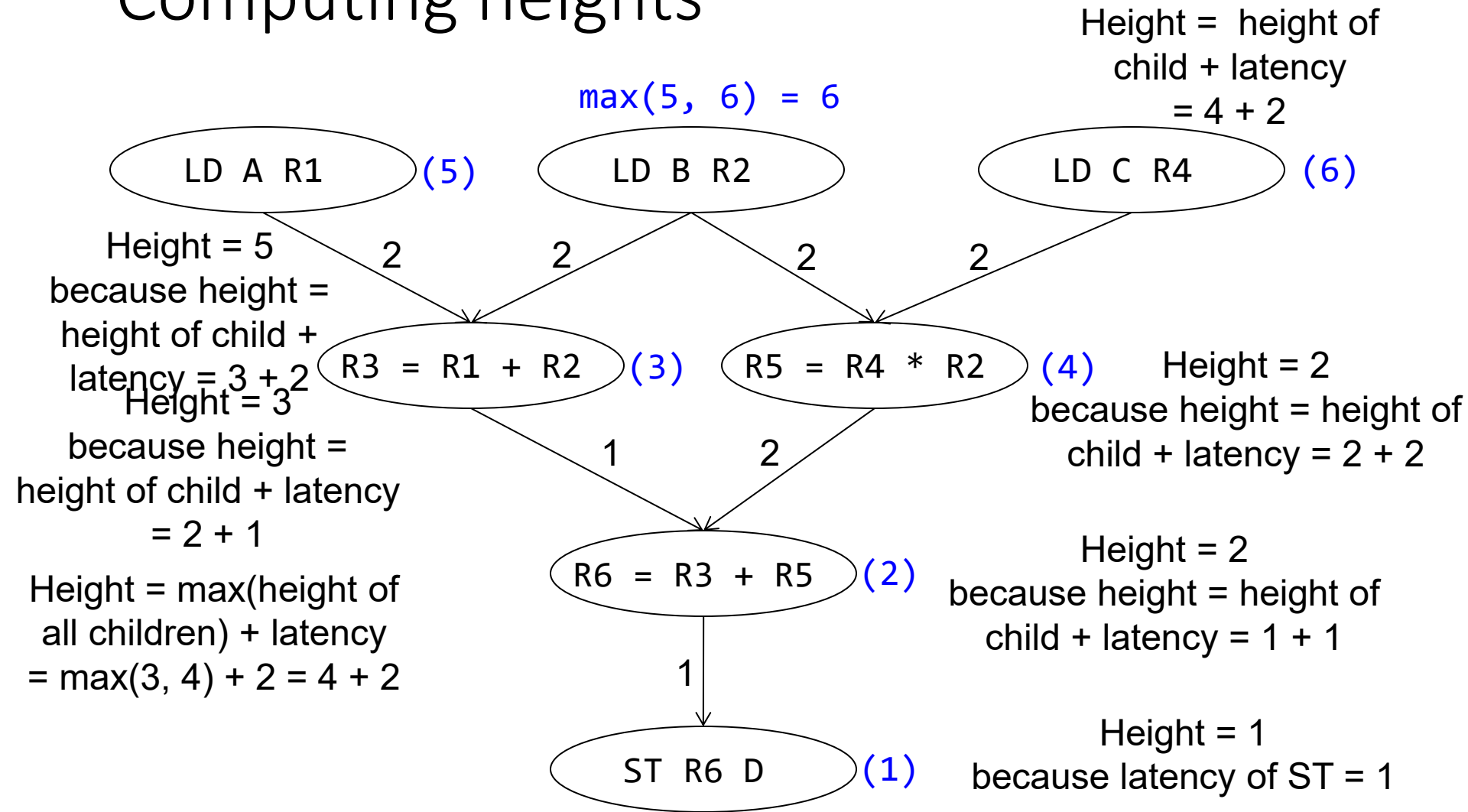
Autumn 2026

Week 9: List Scheduling (contd.), Basic
Blocks, CFG,

Height-based scheduling

- Important to prioritize instructions
 - Instructions that have a lot of downstream instructions dependent on them should be scheduled earlier
- Instruction scheduling NP-hard in general, but **height-based scheduling** is effective
- Instruction height = latency from instruction to farthest-away leaf
 - Leaf node height = instruction latency
 - Interior node height = $\max(\text{heights of children} + \text{instruction latency})$
- Schedule instructions with highest height first

Computing heights



Height-based list scheduling

1. LD A, R1
2. LD B, R2
3. $R3 = R1 + R2$
4. LD C, R4
5. $R5 = R4 * R2$
6. $R6 = R3 + R5$
7. ST R6, D

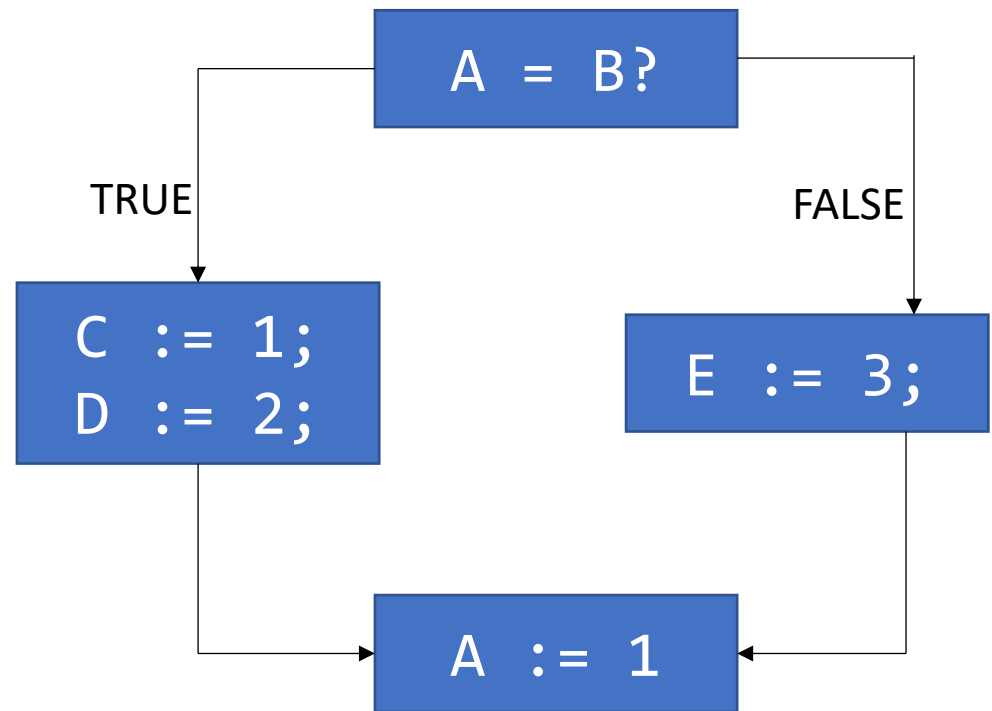
Cycle	ALU0	ALU1	LD/ST
0			2
1			2
2			4
3			4
4	5		1
5			1
6	3		
7	6		
8	7		
9			
10			

Basic Blocks and Flow Graphs

- Basic Block
 - Maximal sequence of consecutive instructions with the following properties:
 - The first instruction of the basic block is the *only entry point*
 - The last instruction of the basic block is either the halt instruction or the *only exit point*
- Flow Graph
 - Nodes are the basic blocks
 - Directed edge indicates which block follows which block

Basic Blocks and Flow Graphs - Example

```
if A = B then  
    C := 1;  
    D := 2;  
else  
    E := 3;  
fi  
A := 1;
```



A data flow graph

Flow Graphs

- Capture how control transfers between basic blocks due to:
 - Conditional constructs
 - Loops
- Are necessary when we want optimize considering larger parts of the program
 - Multiple procedures
 - Whole program

Flow Graphs - Representation

- We need to label and track statements that are jump targets
 - **Explicit targets** – targets mentioned in jump statement
 - **Implicit targets** – targets that follow conditional jump statement
 - Statement that is executed if the branch is not taken
- Implementation
 - Linked lists for Basic Blocks
 - Graph data structures for flow graphs

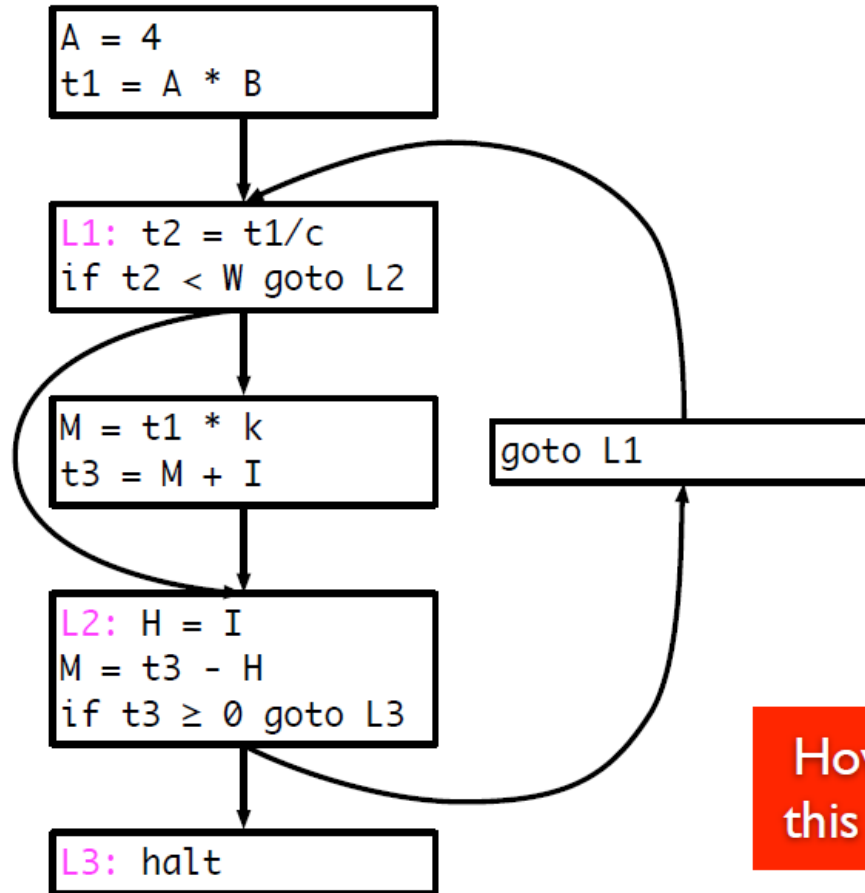
Running example

```
A = 4
t1 = A * B
repeat {
  t2 = t1/C
  if (t2 ≥ W) {
    M = t1 * k
    t3 = M + I
  }
  H = I
  M = t3 - H
} until (T3 ≥ 0)
```

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

CFG for running example



How do we build
this automatically?

Constructing a CFG

- To construct a CFG where each node is a basic block
 - Identify *leaders*: first statement of a basic block
 - In program order, construct a block by appending subsequent statements up to, but not including, the next leader
- Identifying leaders
 - First statement in the program
 - Explicit target of any conditional or unconditional branch
 - Implicit target of any branch

Partitioning algorithm

- Input: set of statements, $stat(i)$ = i^{th} statement in input
- Output: set of *leaders*, set of basic blocks where $block(x)$ is the set of statements in the block with leader x
- Algorithm

```
leaders = {1}           //Leaders always includes first statement
for i = 1 to |n|       //|n| = number of statements
    if  $stat(i)$  is a branch, then
        leaders = leaders  $\cup$  all potential targets
    end for
worklist = leaders
while worklist not empty do
    x = remove earliest statement in worklist
    block(x) = {x}
    for (i = x + 1; i  $\leq$  |n| and i  $\notin$  leaders; i++)
        block(x) = block(x)  $\cup$  {i}
    end for
end while
```

Running example

```

1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt

```

Leaders = ?

Basic blocks = ?

```

leaders = {1}      //Leaders always includes first statement
for i = 1 to |n|    //|n| = number of statements
    if stat(i) is a branch, then
        leaders = leaders ∪ all potential targets
end for
worklist = leaders

```

worklist = leaders

Running example

1	A = 4
---	-------

```
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11}
Basic blocks = ?

worklist = leaders

while *worklist* not empty **do**

x = remove earliest statement in *worklist*

block(x) = {*x*}

for (*i* = *x* + 1; *i* ≤ |*n*| and *i* ∉ *leaders*; *i*++)

block(x) = *block(x)* ∪ {*i*}

end for

end while

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11}

Basic blocks =

Block(1) = ?

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11}

Basic blocks =

Block(1) = ?

Start from statement 2 and add
till either the end or a leader is
reached

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(1) = {1, 2}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(3) = ?

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(3) = {3, 4}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(5) = ?

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(5) = {5, 6}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(7) = ?

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(7) = {7, 8, 9}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(10) = ?
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(10) = {10}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(11) = {11}
Basic blocks =

Running example

1		A = 4
2		t1 = A * B
3	L1:	t2 = t1 / C
4		if t2 < W goto L2
5		M = t1 * k
6		t3 = M + I
7	L2:	H = I
8		M = t3 - H
9		if t3 ≥ 0 goto L3
10		goto L1
11	L3:	halt

Leaders = {1, 3, 5, 7, 10, 11}

Basic blocks = { {1, 2}, {3, 4}, {5, 6}, {7, 8, 9}, {10}, {11} }

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```
for  $i = 1$  to  $|block|$      $\{\{1,2\}, \{3,4\}, \{5,6\}, \{7,8,9\}, \{10\}, \{11\}\}$   
     $x =$  last statement of  $block(i)$   
    if  $stat(x)$  is a branch, then  
        for each explicit target  $y$  of  $stat(x)$   
            create edge from block  $i$  to block  $y$   
        end for  
    if  $stat(x)$  is not unconditional then  
        create edge from block  $i$  to block  $i+1$   
    end for
```

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```
for  $i = 1$  to  $|block|$      $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$   
     $x = \text{last statement of } block(i)$     2:  $t1 = A * B$   
    if  $stat(x)$  is a branch, then  
        for each explicit target  $y$  of  $stat(x)$   
            create edge from block  $i$  to block  $y$   
        end for  
    if  $stat(x)$  is not unconditional then  
        create edge from block  $i$  to block  $i+1$   
end for
```

Edge from block 1 to block 2

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```
for  $i = 1$  to  $|block|$      $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$   
     $x = \text{last statement of } block(i)$     4: if  $t2 < W$  goto L2  
    if  $stat(x)$  is a branch, then  
        for each explicit target  $y$  of  $stat(x)$     Edge from block 2 to block 4  
            create edge from block  $i$  to block  $y$   
        end for  
    if  $stat(x)$  is not unconditional then  
        create edge from block  $i$  to block  $i+1$   
    end for
```

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

for $i = 1$ to $|block|$ $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$

$x = \text{last statement of } block(i)$

if $stat(x)$ is a branch, **then**

for each explicit target y of $stat(x)$

 create edge from block i to block y

end for

if $stat(x)$ is not unconditional **then**

 create edge from block i to block $i+1$

end for

Edge from block 2 to block 3

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG



```

for  $i = 1$  to  $|block|$ 
     $x = \text{last statement of } block(i)$ 
    if  $stat(x)$  is a branch, then
        for each explicit target  $y$  of  $stat(x)$ 
            create edge from block  $i$  to block  $y$ 
        end for
    if  $stat(x)$  is not unconditional then
        create edge from block  $i$  to block  $i+1$ 
    end for
    
```

6: $t3 = M + I$

Edge from block 3 to block 4

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

for $i = 1$ to $|block|$ $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$

$x = \text{last statement of } block(i)$

if $stat(x)$ is a branch, **then**

for each explicit target y of $stat(x)$

 create edge from block i to block y

end for

if $stat(x)$ is not unconditional **then**

 create edge from block i to block $i+1$

end for

Edge from block 4 to block 6



Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

for $i = 1$ to $|block|$ $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$

$x = \text{last statement of } block(i)$

if $stat(x)$ is a branch, **then**

for each explicit target y of $stat(x)$

 create edge from block i to block y

end for

if $stat(x)$ is not unconditional **then**

 create edge from block i to block $i+1$

end for

Edge from block 4 to block 5

Putting edges in CFG

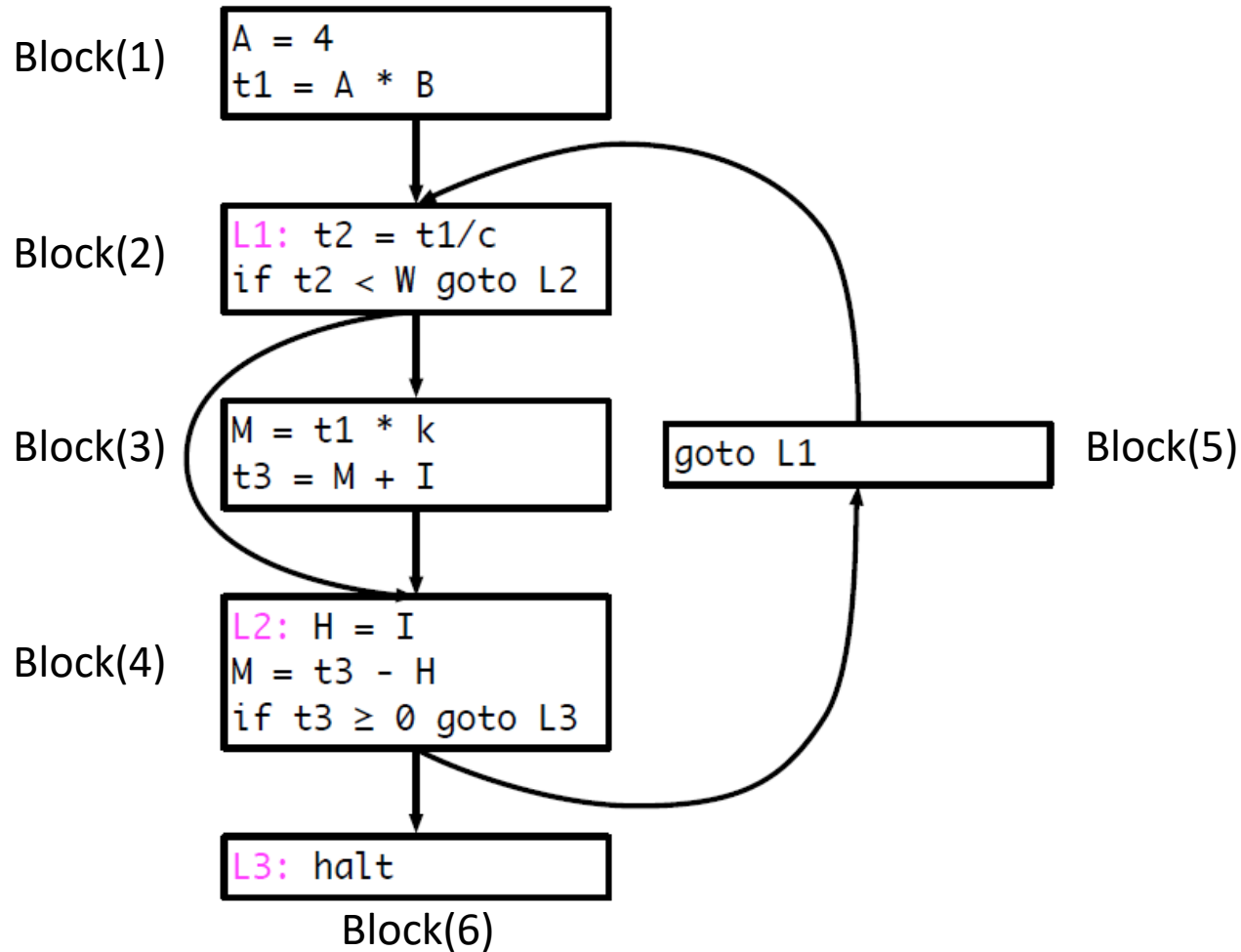
- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```

for  $i = 1$  to  $|block|$      $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$ 
     $x = \text{last statement of } block(i)$ 
    if  $stat(x)$  is a branch, then
        for each explicit target  $y$  of  $stat(x)$ 
            create edge from block  $i$  to block  $y$ 
        end for
    if  $stat(x)$  is not unconditional then
        create edge from block  $i$  to block  $i+1$ 
    end for
    
```

Edge from block 5 to block 2

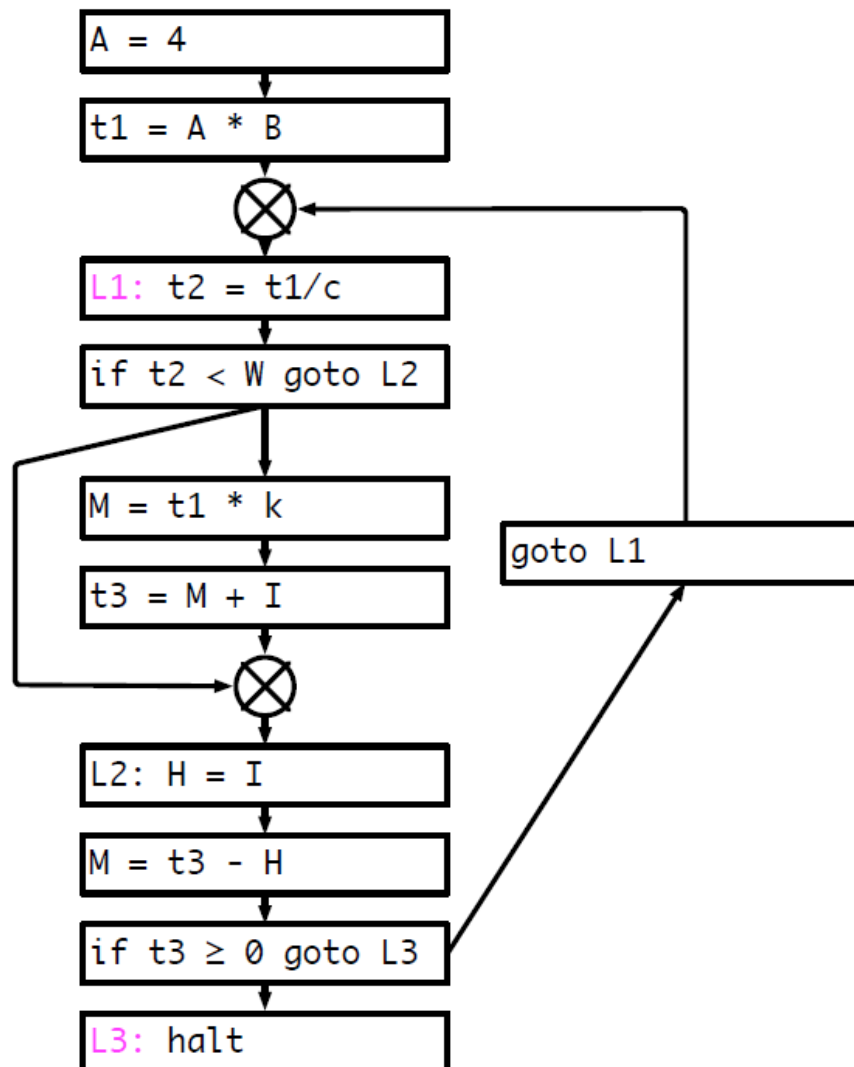
Result



Discussion

- Some times we will also consider the *statement-level* CFG, where each node is a statement rather than a basic block
- Either kind of graph is referred to as a CFG
- In statement-level CFG, we often use a node to explicitly represent *merging* of control
- Control merges when two different CFG nodes point to the same node
- Note: if input language is *structured*, front-end can generate basic block directly
- “GOTO considered harmful”

Statement level CFG



Control Flow Graphs - Use

- Why do we need CFGs? - Global Optimization
 - Optimizing compilers do global optimization (i.e. optimize beyond basic blocks)
 - Differentiating aspect of normal and optimizing compilers
 - E.g. loops are the most frequent targets of global optimization (because they are often the “hot-spots” during program execution)

how do we identify loops in CFGs?